

Récapitulatif de l'itération 1

Planification

Points d'amélioration

- **Estimations** : Les estimations initiales étaient optimistes et n'ont pas suffisamment pris en compte les erreurs d'implémentation potentielles ni les retours en arrière nécessaires.
- **Organisation des réunions** : Difficulté à maintenir des réunions complètes et concises, malgré le format debout adopté.
- **Pair programming** : L'efficacité du pair programming pourrait être améliorée (plus de variété dans les binômes)
- **Qualité du code** : Le code produit peut être optimisé.
- **Gestion des tâches** : Les tâches étaient trop vastes et statiques, ce qui a limité la fréquence des merges et complexifié les rebases. La prise en compte du refactoring et l'apprentissage de nouvelles bibliothèques (JavaFX) n'ont pas été suffisamment anticipés.
- **Avancée des histoires** :
 - Histoire 9 : non implémentée
 - Histoires 7 et 8 : non terminées

Points positifs

- **Attribution des tâches** : L'assignation des tâches entre collaborateurs s'est déroulée efficacement.
- **Communication** : Excellente communication d'équipe et bonne utilisation des outils GitLab.
- **Documentation** : Création de documents, templates et guides pour standardiser les tâches récurrentes (ex : création de widgets de vue).
- **Merge request et review** : Le fait d'effectuer des merges via les merges request GitLab nous a permis :
 - Être plus au fait de l'avancée du code.
 - Faire des reviews entre collaborateurs.

Choix de conceptions

MVC

Nous avons appliqué le pattern MVC, exactement identique à celui vu dans le TP3 intitulé MVC avancé :

- **MetaController** qui gère les autres controllers et la fenêtre affichée.
- Un controller spécifique pour chaque fenêtre et sa view associée.

Message Controller vers View Nous utilisons des DTOs afin d'éviter que la view manipule directement des objets du model.

Notre implémentation : **BugemonDTO** est une interface pour la classe **Bugemon**.

Message View vers Controller Initialement, lorsqu'une vue devait envoyer un message à son controller, concernant bugemon, celle-ci passait un **BugemonDTO** en argument.

Ceci requerrait ensuite que le controller *cast* le **BugemonDTO** reçu en un **Bugemon** pour pouvoir le passer à des méthodes du model.

Nous pensons que ceci était un mauvais choix d'implémentation, car il faut minimiser les *casts* vers le haut dans la hiérarchie.

Solution : nous avons refactor pour utiliser les IDs des bugemons dans ces situations où la view communique vers son controller.

Nous utilisons directement les IDs des bugemons pour ne pas devoir *cast* des `BugemonDTO` en `Bugemons`.

Composants réutilisables (View)

Nous avons commencé par écrire l'écran permettant de créer son équipe, comprenant :

- une grille affichant tous les bugemons disponibles
- une grille affichant les bugemons dans l'équipe

Initialement, tout le code lié à ces deux grilles se trouvait dans la classe gérant la view de cet écran et était entièrement couplé à celle-ci.

Nous avons réalisé que ceci introduisait de la duplication de code dans les autres écrans qui devaient afficher ces mêmes grilles.

Solution : refactoring pour séparer le reste du code de chacune de ces grilles dans des composants séparés.

Bugemon Builder

Utilisation du builder pattern pour faciliter la création de Bugemon.

Note : celui-ci impose de set un ID au bugemon, mais les autres paramètres ont des valeurs par défaut. Ceci nous a permis de progresser plus rapidement dans le GUI, car plus facile pour hardcoder des Bugemons.

Factory

`createRandomTeam` permet de créer une équipe aléatoire de Bugemon.

Types de retour de fonctions : Optional vs Exception

Les règles que nous utilisons pour déterminer le type de retour entre `Optional<T>` VS `T` + exceptions :

Type de fonction	Type de retour et exception
fonction qui pourrait ne pas avoir de résultat (sans que cela soit anormal)	<code>Optional<T></code> (pas d'exception)
fonction qui pourrait rater (cas exceptionnel)	<code>T</code> mais peut throw une exception